

Estimación de TDEW: Pseudocódigo PRAM y Complejidad

Pipeline de regresión lineal local sobre anomalías con diseño bucket-año

Piero Palacios

2026-05-07

Table of contents

1 Propósito	1
2 Modelo PRAM	2
3 Símbolos	2
4 Cómo Leer las Fórmulas de Complejidad	3
5 Algoritmo 1: Pipeline Completo de TDEW	4
6 Algoritmo 2: Construir Climatología	5
7 Algoritmo 3: Construir el Dataset de Entrenamiento Bucket-Año	6
8 Algoritmo 4: Fragmentar Climatología por Bucket	7
9 Algoritmo 4b: Fragmentar TMIN Futuro por Bucket	8
10 Algoritmo 5: Entrenar Coeficientes de Anomalía	8
11 Algoritmo 6: Verificar Completitud	11
12 Algoritmo 7: Reparar DOYs Fallidos	12
13 Algoritmo 8: Parchar Coeficientes	12
14 Algoritmo 9: Pronóstico Opcional (bucketizado)	13
15 Diseño Antiguo vs Diseño Bucket-Año	14
16 Tabla Resumen de Complejidad	15
17 Complejidad Global	15
18 Interpretación HPC	17
19 Notas Prácticas para Esta Máquina	18
19.1 El hardware es simplemente más procesadores	18
20 Metodología Experimental	19
20.1 Cantidades medidas	19
20.2 Definición de un “procesador” p	19
20.3 Controles	19
20.4 Escalamiento fuerte (problema fijo, más procesadores)	20
20.5 Escalamiento débil (más procesadores, problema proporcionalmente mayor)	20
20.6 Datasets: PISCOt v1.0 vs v2.0	20
20.7 Configuraciones de hardware	20
20.8 Reportes	21

1 Propósito

Este documento describe el algoritmo completo utilizado para estimar la temperatura diaria de punto de rocío (TD) a partir de la temperatura mínima (TMIN) para muchas ubicaciones espaciales (ID). El diseño sigue una visión al estilo PRAM: las operaciones independientes se marcan con **pardo**.

El proyecto utiliza un modelo de regresión lineal local sobre anomalías:

$$TD_{anom}(t) = TD(t) - TD_{clim}(ID, doy)$$

$$TMIN_{anom}(t) = TMIN(t) - TMIN_{clim}(ID, doy)$$

Para cada **ID** espacial y cada día del año **doy**, el modelo ajusta una regresión lineal ponderada usando la anomalía de **TMIN** y variables de anomalía rezagadas.

Por qué anomalías, en términos sencillos. La temperatura cruda tiene un ciclo estacional fuerte y predecible: cada ubicación es más cálida en verano y más fría en invierno. Si modeláramos **TD** cruda contra **TMIN** cruda, la mayor parte de la relación aparente sería simplemente “ambas siguen las estaciones” — no el vínculo físico que nos interesa. Por eso primero restamos el valor *típico* de cada ubicación para ese día del año (su **climatología**), dejando la **anomalía**: cuánto está hoy por encima o por debajo de lo normal. La regresión aprende entonces el acoplamiento genuino día a día entre el punto de rocío y la temperatura mínima, habiendo eliminado la estación. Ajustamos una regresión pequeña *separada* para cada (**ID**, **doy**), de modo que la relación pueda cambiar por ubicación y por época del año, y tomamos fuerza prestada de los días vecinos del año (una ventana local de semianchura **h**) para que cada ajuste tenga suficientes muestras. Los productos finales son: una tabla de climatología, los coeficientes de regresión ajustados por (**ID**, **doy**) y, opcionalmente, valores pronosticados de **TD**.

2 Modelo PRAM

PRAM (Máquina de Acceso Aleatorio Paralela, por sus siglas en inglés) es el modelo idealizado estándar para razonar sobre algoritmos paralelos: muchos procesadores comparten una memoria, y contamos cómo se reparte el trabajo entre ellos. Usamos la variante **CREW** (“Lectura Concurrente, Escritura Exclusiva”), que se ajusta exactamente a este pipeline:

- **Lectura concurrente**: muchos procesadores pueden leer las *mismas* tablas de entrada compartidas a la vez (p. ej., cada trabajador puede leer la climatología). Las lecturas no entran en conflicto.
- **Escritura exclusiva**: no hay dos procesadores que escriban en la *misma* ubicación de salida. Aquí esto se garantiza de forma natural — cada trabajador es dueño de un bucket / ID / día del año de salida disjunto, así que nunca hay colisiones de escritura que coordinar.
- **Reducciones** (sumas, medias, conteos) se tratan como reducciones paralelas: combinar **m** valores cuesta aproximadamente $\log m$ pasos en lugar de **m**, porque los valores se suman por pares ascendiendo por un árbol.
- **pardo** marca un bucle cuyas iteraciones son independientes y, por tanto, pueden ejecutarse en paralelo. Es la anotación central de este documento: dondequiera que veas **pardo**, no hay dependencia entre iteraciones, así que un planificador es libre de repartirlas entre todos los trabajadores disponibles.

El diseño bucket-año es importante porque evita que 300 mil tareas independientes por ID escaneen repetidamente los mismos archivos parquet históricos. En lugar de que cada ID vuelva a abrir cada archivo mensual, los datos se agrupan una sola vez en **B** buckets, y cada trabajador lee los archivos compactos de su bucket una única vez — convirtiendo una tormenta de lecturas diminutas y repetidas en unas pocas lecturas secuenciales grandes.

3 Símbolos

Símbolo	Significado
N	Número de ubicaciones espaciales, aproximadamente 300,000 IDs
Y	Número de años de entrenamiento, aproximadamente 40
D	Días por año, como máximo 366
T	Total de días por ID, $T = Y * D$

Símbolo	Significado
R	Total de filas por variable climática, $R = N * T$
B	Número de buckets de ID, por ejemplo 1024
p	Número de procesadores o ranuras de trabajador disponibles
h	Semiventana local de DOY, por defecto $h = 11$
W	Muestras por regresión local, $W = O(Y * h)$
F	Número de características de regresión más el intercepto, constante aquí
Q	Número de DOYs fallidos a reparar, $Q \leq D$
H	Horizonte de pronóstico en días

En este proyecto, D , h y F son constantes pequeñas en la práctica, pero se muestran explícitamente en las fórmulas de complejidad.

4 Cómo Leer las Fórmulas de Complejidad

Cada algoritmo a continuación se describe con tres tipos de costo. Leerlos es más fácil si se tiene presente su significado en lenguaje llano:

- **Trabajo W (costo secuencial).** El número *total* de operaciones básicas que realiza el algoritmo, sin importar cuántos procesadores lo ejecuten. Es lo que se pagaría en un solo procesador. Nunca disminuye al añadir hardware — es el tamaño intrínseco del cómputo.
- **Tiempo paralelo T_p (tiempo de reloj con p procesadores).** Cuánto tarda el algoritmo cuando p trabajadores se ejecutan en paralelo. La forma recurrente es

$$T_p \approx \underbrace{\frac{W}{p_{eff}}}_{\text{trabajo repartido}} + \underbrace{\text{sobrecosto}}_{\text{coordinación, E/S, dependencias}}.$$

El primer término es el trabajo dividido entre los trabajadores (aceleración perfecta). El segundo término es lo que impide que la aceleración sea perfecta: combinar resultados parciales (una *reducción*, que cuesta cerca del $\log$ del número de elementos), leer y escribir archivos, contabilidad del planificador y cualquier paso que deba esperar a uno anterior.

- **Span T_∞ (el camino crítico).** El tiempo incluso con procesadores *ilimitados* — la longitud de la cadena más larga de pasos que deben ocurrir en orden. Un span pequeño significa que el algoritmo es “ancho” (altamente paralelo); un span grande significa que algo es inherentemente secuencial.

Dos notaciones se repiten y vale la pena leerlas literalmente:

- Un término $+ \log(\dots)$ es el costo de una **reducción** paralela — p. ej., sumar muchos números en uno solo por sumas pareadas repetidas toma cerca de $\log$ pasos. Es pequeño pero no nulo, razón por la cual T_p nunca alcanza exactamente W/p .
- $\min(p, B)$ aparece dondequiera que el trabajo se corte en B **tareas de bucket** independientes. Solo se pueden ejecutar a la vez tantos buckets como trabajadores se tengan y como buckets existan, así que el paralelismo utilizable es $\min(p, B)$. Si $p > B$, los trabajadores sobrantes no tienen bucket que tomar y quedan ociosos — el origen de la regla $p_{\{eff\}} \leq B$ que se discute en la sección de HPC.

Así que, en una frase: **W dice cuán grande es el cómputo, T_p dice cuánto tiempo de reloj puede recuperar el clúster, y los términos de sobrecosto dicen por qué esa aceleración acaba deteniéndose.**

5 Algoritmo 1: Pipeline Completo de TDEW

```
1  Algorithm TDEW-PIPELINE
2  Input:
3    TD monthly files for all IDs and dates
4    TMIN monthly files for all IDs and dates
5    Grid table containing the expected IDs
6    Parameters: B buckets, h local DOY window, p processors
7
8  Output:
9    Daily climatology per (ID, doy)
10   Bucket-year merged training dataset
11   Regression coefficients per (ID, doy)
12   Optional repair coefficients for incomplete DOYs
13   Optional TD predictions
14
15  1  climatology <- BUILD-CLIMATOLOGY(TD, TMIN)
16  2  bucketed_training <- BUILD-BUCKET-YEAR-DATASET(TD, TMIN, B)
17  3  bucketed_climatology <- SHARD-CLIMATOLOGY(climatology, B)
18  4  coefficients <- TRAIN-COEFFICIENTS(bucketed_training,
19                                     bucketed_climatology,
20                                     h,
21                                     B)
22  5  incomplete_doy <- CHECK-COMPLETENESS(coefficients, Grid)
23  6  if incomplete_doy is not empty then
24  7    patch <- RERUN-FAILED-DOYS(bucketed_training,
25                                bucketed_climatology,
26                                incomplete_doy,
27                                h,
28                                B)
29  8    coefficients <- PATCH-COEFFICIENTS(coefficients,
30                                         patch,
31                                         incomplete_doy)
32  9    incomplete_doy <- CHECK-COMPLETENESS(coefficients, Grid)
33 10 end if
34 11 if forecasting is requested then
35 12   bucketed_future_tmin <- SHARD-FUTURE-TMIN(TMIN future, B)
36 13   predictions <- FORECAST-TD(coefficients,
37                                bucketed_climatology,
38                                bucketed_training,
39                                bucketed_future_tmin,
40                                H,
41                                B)
42 14 end if
43 15 return coefficients, predictions
```

Lectura del pipeline. El flujo es: primero aprender los valores normales estacionales de cada ubicación (**climatología**, línea 1); reorganizar los archivos mensuales crudos en archivos compactos por bucket para que las etapas posteriores lean de forma eficiente (**construir dataset bucket-año** y **fragmentar climatología**, líneas 2–3); ajustar las regresiones locales que son la salida científica (**entrenar coeficientes**, línea 4); luego un bucle de control de calidad (líneas 5–10) que verifica si cada día del año se ajustó con éxito para cada

ubicación y, si no, reentrena *solo* los días faltantes y los integra; y finalmente un **pronóstico** opcional (líneas 11–14) que avanza el modelo ajustado hacia adelante sobre un horizonte futuro. Los pasos se ejecutan en secuencia, pero cada paso es internamente paralelo (sus bucles *pardo*). Los algoritmos posteriores expanden cada una de estas líneas.

6 Algoritmo 2: Construir Climatología

La climatología es la media histórica para cada (ID, doy).

```

1  Algorithm BUILD-CLIMATOLOGY
2  Input:
3      TD rows: (ID, date, TD)
4      TMIN rows: (ID, date, TMIN)
5
6  Output:
7      climatology rows: (ID, doy, TD_clim, TMIN_clim)
8
9  1  for each climate variable V in {TD, TMIN} pardo do
10  2      for each monthly file m of V pardo do
11  3          read rows from file m
12  4          for each row r pardo do
13  5              r.doy <- day_of_year(r.date)
14  6              emit partial record (r.ID, r.doy, r.value)
15  7          end for
16  8      end for
17  9      for each key (ID, doy) pardo do
18  10          sum_V(ID, doy) <- parallel sum of values with this key
19  11          count_V(ID, doy) <- parallel count of values with this key
20  12          clim_V(ID, doy) <- sum_V(ID, doy) / count_V(ID, doy)
21  13      end for
22  14 end for
23  15 join TD_clim and TMIN_clim by (ID, doy)
24  16 return climatology

```

Complejidad:

$$W_{clim} = O(R)$$

$$T_p = O(R/p + \log R)$$

Espacio:

$$S_{clim} = O(ND)$$

Qué significan los costos. El trabajo $O(R)$ indica que tocamos cada fila de cada variable climática exactamente una vez — no hay forma de calcular promedios sin mirar todos los datos, así que esto es tan barato como puede serlo. El tiempo paralelo $O(R/p + \log R)$ indica que el recorrido de filas se reparte perfectamente entre p trabajadores (R/p), más una pequeña cola $\log R$ para combinar las sumas parciales por clave en medias finales (la reducción). La salida es un número por par (ID, doy), de ahí la memoria $O(ND)$ — mucho menor que la entrada cruda, porque 40 años de valores diarios colapsan en una sola curva estacional por ubicación.

7 Algoritmo 3: Construir el Dataset de Entrenamiento Bucket-Año

Este es el rediseño principal. La costosa fusión mensual de TD/TMIN se paga una sola vez. Las filas se distribuyen luego a buckets de ID deterministas y se compactan por año.

```
1  Algorithm BUILD-BUCKET-YEAR-DATASET
2  Input:
3    TD monthly files
4    TMIN monthly files
5    Number of buckets B
6
7  Output:
8    bucket-year training files:
9    bucket b, year y -> rows (ID, date, TD, TMIN, doy)
10
11 1  for each year y pardo do
12 2    for each month m in year y pardo do
13 3      TD_m <- read TD rows for month m
14 4      TMIN_m <- read TMIN rows for month m
15 5      M_m <- hash join TD_m and TMIN_m by (ID, date)
16 6      for each row r in M_m pardo do
17 7        r.doy <- day_of_year(r.date)
18 8        r.bucket <- r.ID mod B
19 9        emit r to temporary partition (bucket = r.bucket, year = y)
20 10     end for
21 11   end for
22 12   for each bucket b in 0..B-1 pardo do
23 13     read all temporary rows for (bucket b, year y)
24 14     sort rows by (ID, date)
25 15     write compact file for (bucket b, year y)
26 16   end for
27 17 end for
28 18 return bucket-year training dataset
```

Complejidad con joins por hash:

$$W_{bucket} = O(R)$$

Si se incluye el ordenamiento dentro de cada fragmento bucket-año:

$$W_{bucket-sort} = O\left(R + \sum_{b,y} n_{b,y} \log n_{b,y}\right)$$

Con buckets balanceados, $n_{\{b,y\}}$ approx $(N/B)D$, así que:

$$W_{bucket-sort} = O\left(R + BY \cdot \frac{ND}{B} \log\left(\frac{ND}{B}\right)\right) = O\left(R \log\left(\frac{ND}{B}\right)\right)$$

Tiempo paralelo ideal:

$$T_p = O\left(\frac{R}{p} + \log\left(\frac{ND}{B}\right)\right)$$

Memoria pico por compactación bucket-año:

$$S_{\text{bucket-year}} = O\left(\frac{ND}{B}\right)$$

La idea en una imagen. Un *bucket* es simplemente un grupo determinista de IDs: $\text{bucket} = \text{ID} \bmod B$. Pagamos la costosa fusión $\text{TD} \leftrightarrow \text{TMIN}$ una sola vez, luego archivamos cada fila en su bucket (y año). Después, cualquier etapa que necesite los datos de un ID lee un único archivo pequeño, ya fusionado y ya ordenado, en lugar de reescanear todo el archivo histórico.

Lectura de las fórmulas. El trabajo $O(R)$ indica de nuevo que pasamos por cada fila una vez. El factor $\log(ND/B)$ que aparece cuando se incluye el ordenamiento es el costo de ordenar las filas *dentro* de cada fragmento bucket-año; como cada fragmento contiene solo unas ND/B filas (no las R totales), ese log es de una cantidad pequeña, así que ordenar añade muy poco. El tiempo paralelo tiene el mismo reparto de trabajo R/p más esa pequeña reducción $\log(ND/B)$. Crucialmente, la memoria pico es $O(ND/B)$, **no** $O(R)$: un trabajador solo mantiene un fragmento bucket-año a la vez, así que aumentar B *reduce* la memoria que cada trabajador necesita. Esa es la palanca que evita que 300 mil IDs $\times$ 40 años tengan que caber en la RAM de una sola máquina.

8 Algoritmo 4: Fragmentar Climatología por Bucket

```

1  Algorithm SHARD-CLIMATOLOGY
2  Input:
3    climatology rows (ID, doy, TD_clim, TMIN_clim)
4    Number of buckets B
5
6  Output:
7    bucketed climatology files
8
9  1  for each climatology row c pardo do
10  2    c.bucket <- c.ID mod B
11  3    emit c to bucket c.bucket
12  4  end for
13  5  for each bucket b in 0..B-1 pardo do
14  6    write climatology rows for bucket b
15  7  end for
16  8  return bucketed climatology

```

Complejidad:

$$W_{\text{clim-shard}} = O(ND)$$

$$T_p = O(ND/p + \log B)$$

Memoria pico con streaming:

$$S_{\text{clim-shard}} = O(ND/B)$$

Si se carga toda la climatología de una vez, la memoria pico se vuelve $O(ND)$.

Por qué existe este paso. La fragmentación no hace aritmética alguna — solo *mueve* cada fila de climatología al bucket al que pertenece su ID, de modo que la etapa de entrenamiento encuentre los valores normales de un ID en el mismo bucket que sus filas de entrenamiento y lea ambos en una sola pasada. El costo es por tanto una única pasada en streaming (trabajo $O(ND)$) más un término $\log B$ para enrutar filas a B destinos; el streaming mantiene en memoria solo el equivalente a un bucket ($O(ND/B)$) a la vez.

9 Algoritmo 4b: Fragmentar TMIN Futuro por Bucket

El pronóstico necesita la **TMIN** futura exógena sobre el horizonte de predicción dispuesta por los mismos buckets que los coeficientes, de modo que cada tarea de pronóstico lea sus entradas una sola vez. Esto refleja el preprocesamiento bucket-año del Algoritmo 3, pero para una sola variable y solo sobre el horizonte de pronóstico.

Sea $R_{fut} = N * H$ el número de filas de TMIN futura sobre el horizonte H .

```
1  Algorithm SHARD-FUTURE-TMIN-BY-BUCKET
2  Input:
3    future TMIN monthly files over the forecast horizon
4    Number of buckets B
5
6  Output:
7    bucketed future-TMIN files: bucket b -> rows (ID, date, TMIN)
8
9  1  for each month m in the horizon pardo do
10  2    read future TMIN rows for month m
11  3    for each row r pardo do
12  4      r.bucket <- r.ID mod B
13  5      emit r to temporary partition (bucket = r.bucket)
14  6    end for
15  7  end for
16  8  for each bucket b in 0..B-1 pardo do
17  9    write compact future-TMIN file for bucket b
18  10 end for
19  11 return bucketed future TMIN
```

Complejidad:

$$W_{fut-shard} = O(R_{fut}) = O(NH)$$

$$T_p = O(NH/p + \log B)$$

Memoria pico con streaming:

$$S_{fut-shard} = O(NH/B)$$

En resumen. Este es el mismo paso de enrutamiento que el Algoritmo 4, pero aplicado a la **TMIN** futura que impulsa el pronóstico, y solo sobre los H días de pronóstico en lugar de toda la historia. Disponerla por los *mismos* buckets significa que cada tarea de pronóstico leerá después sus coeficientes, climatología, historia y **TMIN** futura, todo desde un mismo bucket, una sola vez.

10 Algoritmo 5: Entrenar Coeficientes de Anomalía

Para cada **ID** y cada **doy** objetivo, ajustar una regresión ponderada sobre una vecindad circular de días del año.

Qué hace realmente un ajuste. Supongamos que queremos el modelo para la ubicación i en el día del año d (digamos el día 60). No usamos solo datos del día 60 — un solo día a lo largo de 40 años son demasiados pocos puntos. En su lugar, reunimos cada observación cuyo día del año esté dentro de h días de d (la *ventana local*), envolviendo en el fin de año de modo que el día 365 sea vecino del día 1 (distancia *circular*). Los puntos más cercanos al día d reciben más influencia mediante un **peso tricube** que cae suavemente a cero en el borde de la ventana, así que el ajuste refleja “alrededor del día 60” sin dejar de estar centrado en el día 60. Sobre esa vecindad ponderada resolvemos una pequeña regresión por mínimos cuadrados ponderados de la

anomalía de TD sobre la anomalía de TMIN y unas pocas anomalías rezagadas. El resultado — los coeficientes de regresión — se almacena para (i, d). Repetimos esto para cada día del año y cada ubicación, razón por la cual el conteo total de ajustes es enorme aunque cada uno sea diminuto.

```

1  Algorithm TRAIN-COEFFICIENTS
2  Input:
3    bucket-year training dataset
4    bucketed climatology
5    Local DOY half-window h
6    Number of buckets B
7
8  Output:
9    coefficients per (ID, doy)
10
11 1  for each bucket b in 0..B-1 pardo do
12 2    train_b <- read all bucket-year training rows for bucket b
13 3    clim_b <- read climatology rows for bucket b
14 4    for each ID i in bucket b pardo do
15 5      X_i <- rows of train_b for ID i
16 6      C_i <- rows of clim_b for ID i
17 7      sort X_i by date
18 8      for each row t in X_i pardo do
19 9        TD_anom(t) <- TD(t) - TD_clim(i, doy(t))
20 10       TMIN_anom(t) <- TMIN(t) - TMIN_clim(i, doy(t))
21 11     end for
22 12     for each row t in X_i pardo do
23 13       create lag features:
24 14         TD_anom_lag1(t), TD_anom_lag2(t), TMIN_anom_lag1(t)
25 15     end for
26 16     for each target day d in 1..366 pardo do
27 17       N_d <- rows where circular_distance(doy(t), d) <= h
28 18       remove rows in N_d with missing target or missing features
29 19       if size(N_d) >= minimum_sample_count then
30 20         for each row t in N_d pardo do
31 21           weight(t) <- tricube(circular_distance(doy(t), d) / h)
32 22         end for
33 23         beta <- weighted_least_squares(N_d)
34 24         write coefficients for (ID i, doy d)
35 25       end if
36 26     end for
37 27   end for
38 28   write coefficient file for bucket b
39 29   write failure log for bucket b, if needed
40 30 end for
41 31 return coefficient dataset

```

Por regresión local:

$$W_{one-fit} = O(WF^2 + F^3)$$

Aquí F es constante, así que:

$$W_{one-fit} = O(W) = O(Yh)$$

De dónde viene $O(WF^2 + F^3)$. Resolver un ajuste por mínimos cuadrados ponderados implica formar la matriz de ecuaciones normales $F \times F$ $X^T W X$ a partir de W filas de muestra ponderadas (ese es el término WF^2 — cada una de las F^2 entradas de la matriz suma sobre W filas) y luego resolver el sistema lineal $F \times F$ resultante (el término F^3 , el costo de una pequeña factorización matricial). Como el número de características $F = 5$ es una constante fija aquí, ambos términos colapsan a una constante por W , así que **un ajuste cuesta $O(W) = O(Yh)$** — proporcional a cuántas muestras de la vecindad ve.

Todo el entrenamiento de coeficientes:

$$W_{train} = O(NDW) = O(NDYh)$$

Eso es simplemente *un ajuste* ($O(Yh)$) por el *número de ajustes* (N ubicaciones $\times D$ días del año). Con $N \approx 3 \times 10^5$ y $D \approx 366$, eso son aproximadamente 1.1×10^8 ajustes independientes — este producto es el costo dominante del pipeline y el objetivo principal de los experimentos de escalamiento.

Span PRAM ideal si buckets, IDs y DOYs son completamente paralelos:

$$T_{\infty} = O(\log W + F^3)$$

Lectura del span. Si tuviéramos procesadores ilimitados, los 1.1×10^8 ajustes podrían ejecutarse todos a la vez, así que el único tiempo que queda es el camino crítico de *un* ajuste: una reducción $\log W$ para sumar la vecindad en las ecuaciones normales, más la resolución constante F^3 . En otras palabras, el algoritmo es extremadamente “ancho” — casi nada es inherentemente secuencial dentro del entrenamiento. La razón por la que las ejecuciones reales son mucho más lentas que T_{∞} es puramente que tenemos p trabajadores, no infinitos, y que los datos deben leerse del disco.

Con p procesadores:

$$T_p = O(NDYh/p + \log(Yh))$$

La ejecución práctica bucket-paralela tiene a lo sumo B tareas externas, así que:

$$T_p = O\left(\frac{NDYh}{\min(p, B)} + \text{sobrecosto de E/S}\right)$$

Por qué $\min(p, B)$ y no p — y qué significa esto para una GPU. En la implementación, la unidad de planificación paralela es el **bucket**: un bucket es una tarea entregada a un trabajador. Así que el número de buckets corriendo *al mismo tiempo* no puede ser mayor que el número de trabajadores p ni mayor que el número de buckets B — de ahí $\min(p, B)$. Esto es paralelismo a **nivel de bucket** (externo), y es lo único que $\min(p, B)$ acota.

Hay un segundo nivel de paralelismo independiente *dentro* de cada tarea de bucket: los $\approx (N/B) \cdot D$ ajustes individuales 5×5 de ese bucket son mutuamente independientes. Cómo se explota ese nivel interno es exactamente lo que difiere entre CPU y GPU:

- **CPU.** Cada trabajador es un proceso de un solo hilo (BLAS fijado a un hilo), así que recorre los ajustes de su bucket de uno en uno. El nivel interno *no* se paraleliza por trabajador; la aceleración proviene enteramente de ejecutar más buckets a la vez. Aquí p = número de procesos trabajadores ($\approx$ núcleos), y se desean **muchos** buckets ($B \geq 4p$) para que ningún núcleo quede ocioso.
- **GPU (A100).** Cada GPU es un trabajador, así que a nivel de bucket p = número de GPUs (a menudo solo 1–8) y $\min(p, B)$ es pequeño. Pero una GPU no procesa los ajustes de un bucket de uno en uno: ensambla las ecuaciones normales $X^T W X$ para *todos* los $\approx (N/B) \cdot D \approx 10^5$ ajustes del bucket y los resuelve en una sola llamada **batched** (por lotes), repartiéndolos entre miles de núcleos CUDA. Ese ancho de lote interno — del orden de 10^5 ajustes ofrecidos al dispositivo por oleada, despejados en milisegundos — es donde reside la velocidad de la GPU, y **$\min(p, B)$ no lo acota**. La implicación práctica invierte la regla de CPU: una GPU

quiere buckets *lo bastante grandes* como para que cada lote llene el dispositivo, así que no se empuja B muy alto en ejecuciones con GPU.

En ambos casos el trabajo *total* $O(NDYh)$ es idéntico; el hardware solo cambia cómo se llenan los dos niveles de paralelismo. La sección “El hardware es simplemente más procesadores” más abajo lo precisa, y los experimentos de escalamiento miden qué hardware despeja más rápido los 1.1×10^8 ajustes.

Memoria pico por tarea de bucket:

$$S_{train-bucket} = O\left(\frac{NT}{B} + \frac{ND}{B}\right)$$

El primer término es la serie temporal de entrenamiento del bucket. El segundo término es la climatología del bucket.

11 Algoritmo 6: Verificar Completitud

```

1  Algorithm CHECK-COMPLETENESS
2  Input:
3    coefficients
4    expected ID count from grid
5
6  Output:
7    incomplete DOYs
8
9  1  expected_count <- number of IDs in grid
10 2  for each coefficient row c pardo do
11 3    emit key c.doy with value c.ID
12 4  end for
13 5  for each doy d in 1..366 pardo do
14 6    observed_count(d) <- distinct count of IDs emitted for d
15 7    if observed_count(d) < expected_count then
16 8      mark d as incomplete
17 9    end if
18 10 end for
19 11 return incomplete DOYs

```

Sea $C = ND$ el número completo de filas de coeficientes.

$$W_{check} = O(C)$$

$$T_p = O(C/p + \log C)$$

Espacio:

$$S_{check} = O(C)$$

Qué verifica. Un ajuste (ID , doy) puede omitirse si su vecindad tuvo demasiadas pocas muestras válidas (línea 19 del entrenamiento). Este paso es una auditoría de cordura: para cada día del año, cuenta cuántas ubicaciones realmente obtuvieron un coeficiente y lo compara con cuántas dice la grilla que deberían existir. Cualquier día del año al que le falten algunas ubicaciones se marca para reparación. Es una única pasada barata sobre la tabla de coeficientes ($O(C)$), y su salida es diminuta — una breve lista de a lo sumo D días a rehacer.

12 Algoritmo 7: Reparar DOYs Fallidos

Solo se reentrenan los DOYs incompletos. Si Q es pequeño, esto es mucho más barato que reentrenar los 366 DOYs.

```
1  Algorithm RERUN-FAILED-DOYS
2  Input:
3    bucket-year training dataset
4    bucketed climatology
5    incomplete DOYs  $Q$ 
6    Local DOY half-window  $h$ 
7
8  Output:
9    patch coefficient dataset
10
11 1  for each bucket  $b$  in  $0..B-1$  pardo do
12 2    train_b <- read all bucket-year training rows for bucket  $b$ 
13 3    clim_b <- read climatology rows for bucket  $b$ 
14 4    for each ID  $i$  in bucket  $b$  pardo do
15 5      recompute anomalies and lags for ID  $i$ 
16 6      for each failed day  $d$  in  $Q$  pardo do
17 7        fit weighted local regression for (ID  $i$ , doy  $d$ )
18 8        write patch coefficients for (ID  $i$ , doy  $d$ )
19 9      end for
20 10    end for
21 11 end for
22 12 return patch coefficient dataset
```

Complejidad:

$$W_{repair} = O(NQYh)$$

$$T_p = O(NQYh/p + \log(Yh))$$

Por qué la reparación es barata. La reparación es estructuralmente idéntica al entrenamiento (mismos buckets, mismos ajustes ponderados) pero restringida a los Q días marcados en lugar de a todos los D . Así que su costo es el costo del entrenamiento con D reemplazado por Q : $O(NQYh)$. Cuando solo falló un puñado de días ($Q \ll D$), esto es una fracción diminuta de un reentrenamiento completo — rehacemos los pocos días rotos, no los 366.

13 Algoritmo 8: Parchar Coeficientes

```
1  Algorithm PATCH-COEFFICIENTS
2  Input:
3    base coefficient dataset
4    patch coefficient dataset
5    incomplete DOYs  $Q$ 
6
7  Output:
8    corrected coefficient dataset
9
10 1  for each bucket  $b$  in  $0..B-1$  pardo do
```

```

11 2    base_b <- read coefficient rows for bucket b
12 3    patch_b <- read patch rows for bucket b
13 4    base_keep <- rows in base_b whose doy is not in Q
14 5    corrected_b <- union(base_keep, patch_b)
15 6    remove duplicate (ID, doy), keeping the patch row
16 7    sort corrected_b by (ID, doy)
17 8    write corrected coefficients for bucket b
18 9  end for
19 10 return corrected coefficient dataset

```

Complejidad:

$$W_{patch} = O(ND + NQ)$$

$$T_p = O((ND + NQ)/p)$$

Memoria pico por bucket:

$$S_{patch-bucket} = O(ND/B + NQ/B)$$

Qué hace el parchado. En lugar de editar la gran tabla de coeficientes en su lugar, cada bucket se reconstruye a sí mismo: conserva las filas cuyo día del año *no* fue reparado, añade las filas recién reparadas y descarta cualquier (ID, doy) duplicado en favor de la reparación. Esto es pura contabilidad — una lectura, un filtro, una unión, un ordenamiento y una escritura por bucket — así que su costo es solo proporcional al tamaño de la tabla más el pequeño parche, sin nuevo ajuste de modelo.

14 Algoritmo 9: Pronóstico Opcional (bucketizado)

El pronóstico es paralelo entre buckets, y entre IDs dentro de un bucket, pero recursivo a través del tiempo para cada ID: el día t depende de la anomalía de TD predicha previamente, así que el bucle del horizonte no puede paralelizarse dentro de un mismo ID. Como en el entrenamiento, el trabajo se organiza por bucket de modo que cada fragmento de coeficientes, climatología, historia y TMIN futura se lea exactamente una vez. Los fragmentos de TMIN futura los produce el Algoritmo 4b; la historia (usada solo para sembrar los dos primeros rezagos) se lee del dataset de entrenamiento bucket-año.

```

1  Algorithm FORECAST-TD
2  Input:
3    bucketed coefficients per (ID, doy)
4    bucketed climatology per (ID, doy)
5    bucketed history (to seed lags) and bucketed future TMIN
6    forecast horizon H
7    Number of buckets B
8
9  Output:
10   predicted TD for each (ID, date)
11
12 1 for each bucket b in 0..B-1 pardo do
13 2   coeff_b <- read coefficient rows for bucket b
14 3   clim_b <- read climatology rows for bucket b
15 4   hist_b <- read history rows for bucket b
16 5   ftmin_b <- read future TMIN rows for bucket b
17 6   for each ID i in bucket b pardo do

```

```

18 7      initialize lagged TD/TMIN anomalies from hist_b(i)
19 8      for forecast day t from 1 to H do          // secuencial en t
20 9          d <- day_of_year(date_t)
21 10         beta <- coeff_b(i, d)
22 11         x <- current TMIN anomaly and lagged anomaly features
23 12         TD_anom_hat <- beta dot x
24 13         TD_hat <- clim_b(i, d) + TD_anom_hat
25 14         update lagged anomalies with TD_anom_hat and TMIN_anom(t)
26 15         write prediction (ID i, date_t, TD_hat)
27 16     end for
28 17 end for
29 18     write prediction file for bucket b
30 19 end for
31 20 return predictions

```

Complejidad:

$$W_{forecast} = O(NH)$$

Tiempo práctico bucket-paralelo (a lo sumo B tareas externas, recursivo sobre el horizonte dentro de cada ID):

$$T_p = O\left(\frac{NH}{\min(p, B)} + H\right)$$

Por qué el pronóstico tiene un $+ H$ que el entrenamiento no tiene. Los ajustes de entrenamiento son mutuamente independientes, así que se paralelizan completamente. El pronóstico es diferente *dentro* de una ubicación: para predecir el día t alimentamos la anomalía *predicha* del día $t-1$ (las características de rezago). El día 2 necesita el día 1, el día 3 necesita el día 2, y así sucesivamente, así que los H días de pronóstico para un solo ID deben calcularse *en orden* — no pueden repartirse entre trabajadores. Esa cadena inevitable de longitud H es el término $+ H$. Aún paralelizamos completamente entre los millones de *ubicaciones* (y entre buckets), así que el trabajo $O(NH)$ se divide por $\min(p, B)$; solo el avance temporal por ID sigue siendo secuencial. Como con el entrenamiento, la estructura de buckets acota el paralelismo efectivo en $p_{\{eff\}} \leq B$, y en una GPU aplica la misma imagen de dos niveles — las recursiones de muchos IDs avanzan un paso temporal juntas como una operación batched entre los núcleos CUDA.

Memoria pico por tarea de bucket:

$$S_{forecast-bucket} = O\left(\frac{NH}{B} + \frac{ND}{B}\right)$$

15 Diseño Antiguo vs Diseño Bucket-Año

El diseño original al estilo Colab entrenaba una tarea por ID. Cada tarea descubría, abría y filtraba los mismos archivos parquet mensuales.

Sea $S = 12Y$ el número de archivos fuente mensuales por variable.

Sobrecosto de E/S por tarea del diseño antiguo:

$$O(NS)$$

Para $N = 300,000$ y $Y = 40$, esto significa aproximadamente:

$$300,000 \times 480 = 144,000,000$$

intentos de acceso a archivos por familia de variables antes de considerar las lecturas reales de filas.

El diseño bucket-año cambia el patrón de E/S:

- Los archivos fuente se escanean una vez durante el preprocesamiento.
- El entrenamiento lee archivos compactos bucket-año.
- Los trabajadores escriben salidas de buckets disjuntas.
- No se requiere una gran concatenación del lado del driver.

Sobrecosto de escaneo de fuentes nuevo:

$$O(S)$$

Trabajo de entrenamiento del modelo nuevo:

$$O(NDYh)$$

Así que el rediseño no elimina el trabajo estadístico de ajustar muchas regresiones locales, pero sí elimina el escaneo repetido de archivos fuente que hacía lenta y frágil la implementación anterior.

16 Tabla Resumen de Complejidad

La tabla recopila los tres costos para cada fase. Lee cada fila como: “*así de grande es la fase (w), así se encoge su tiempo de reloj con p trabajadores (T_p), y cuánta memoria necesita un trabajador.*” Las fases con $\min(p, B)$ en su T_p (entrenamiento, reparación, pronóstico) son las etapas de cómputo bucket-paralelas cuya aceleración se satura una vez que p alcanza B ; las otras (solo $/p$) son pasadas limitadas por E/S que escalan con los trabajadores hasta que el sistema de archivos se convierte en el límite. La única entrada dominante es **entrenar coeficientes**, $O(NDYh)$.

Fase	Trabajo secuencial	Tiempo paralelo ideal con p procesadores	Objetivo de memoria pico
Construir clima-tología	$O(R)$	$O(R/p + \log R)$	$O(ND)$ salida
Construir dataset bucket-año	$O(R)$ o $O(R \log(ND/B))$ con ordenamiento	$O(R/p + \log(ND/B))$	$O(ND/B)$ por compactación
Fragmentar clima-tología	$O(ND)$	$O(ND/p + \log B)$	$O(ND/B)$ streaming
Fragmentar TMIN futura	$O(NH)$	$O(NH/p + \log B)$	$O(NH/B)$ streaming
Entrenar coeficientes	$O(NDYh)$	$O(NDYh/\min(p, B) + \log(Yh))$	$O(NT/B + ND/B)$ por bucket
Verificar completitud	$O(ND)$	$O(ND/p + \log(ND))$	$O(ND)$ o streaming por bucket
Reparar Q DOYs fallidos	$O(NQYh)$	$O(NQYh/\min(p, B) + \log(Yh))$	igual que el bucket de entrenamiento
Parchar coeficientes	$O(ND + NQ)$	$O((ND + NQ)/p)$	$O(ND/B + NQ/B)$ por bucket
Pronóstico horizonte H	$O(NH)$	$O(NH/\min(p, B) + H)$	$O(NH/B)$ por bucket de salida

17 Complejidad Global

El pipeline completo es una secuencia de fases paralelas, así que el trabajo total es la suma del trabajo de cada fase. Los términos dominantes son:

- preprocesamiento bucket-año,
- entrenamiento completo de coeficientes,
- reparación opcional de DOYs fallidos,
- pronóstico opcional.

Usando $R = NYD$, el trabajo secuencial total para el preprocesamiento más el entrenamiento completo sin reparación y sin pronóstico es:

$$W_{total} = O\left(R \log\left(\frac{ND}{B}\right) + NDYh\right)$$

Equivalentemente:

$$W_{total} = O\left(NYD \log\left(\frac{ND}{B}\right) + NDYh\right)$$

Si el costo de ordenamiento dentro de los archivos bucket-año se ignora o se trata como un detalle de implementación, la expresión más simple es:

$$W_{total} = O(NYD + NDYh) = O(NDYh)$$

porque h es el principal multiplicador de la regresión local.

Si se necesita reparación para Q DOYs fallidos, añadir:

$$W_{repair-total} = O(NQYh)$$

Así que el entrenamiento completo más la reparación es:

$$W_{total+repair} = O\left(NYD \log\left(\frac{ND}{B}\right) + NYh(D + Q)\right)$$

Si también se ejecuta el pronóstico para el horizonte H , añadir:

$$W_{forecast-total} = O(NH)$$

Por tanto, la expresión de extremo a extremo más completa es:

$$W_{end-to-end} = O\left(NYD \log\left(\frac{ND}{B}\right) + NYh(D + Q) + NH\right)$$

Para la ejecución paralela, las fases son secuencialmente dependientes, pero cada fase usa **pardo** internamente. Con p procesadores y B tareas de bucket, el tiempo paralelo práctico es:

$$T_p = O\left(\frac{NYD \log(ND/B)}{p} + \frac{NDYh}{\min(p, B)} + \frac{NQYh}{\min(p, B)} + \frac{NH}{\min(p, B)} + H + \log(NYD)\right)$$

Término por término, ese tiempo paralelo se lee como: construir el dataset bucket-año ($NYD \log(ND/B) / p$), hacer el entrenamiento completo ($NDYh / \min(p, B)$, el grande), reparar los días fallidos si los hay ($NQYh / \min(p, B)$), ejecutar el pronóstico ($NH / \min(p, B)$), pagar la inevitable recursión de pronóstico por ID (+ H), y pagar una reducción global (+ $\log(NYD)$). Los términos de preprocesamiento se dividen por p (están limitados por el sistema de archivos, no por los buckets), mientras que las tres etapas de cómputo se dividen por $\min(p, B)$ (están limitadas por el número de buckets).

El término + H permanece porque el pronóstico es recursivo en el tiempo para cada ID. Si no se ejecuta el pronóstico, eliminar $NH/\min(p, B) + H$. Si no existen DOYs fallidos, eliminar el término $NQYh / \min(p, B)$.

Para este proyecto, el cuello de botella práctico tras el rediseño bucket-año es:

$$O(NDYh)$$

Ese es el costo de ajustar regresiones locales para muchos IDs y todos los días del año. El rediseño reduce principalmente la E/S repetida del antiguo enfoque al estilo Colab.

18 Interpretación HPC

La complejidad de trabajo total no depende de p porque mide el número total de operaciones requeridas por el algoritmo. Los mismos joins, cálculos de anomalías, regresiones locales, reparaciones y pronósticos deben computarse tanto si el trabajo corre en un procesador como si corre en un clúster universitario de cómputo de alto rendimiento.

En un sistema HPC, la medida relevante para el tiempo de ejecución es el tiempo de ejecución paralelo:

$$T_p = O\left(\frac{W}{p_{eff}} + \text{sobrecosto de E/S} + \text{sobrecosto de planificación} + \text{dependencias secuenciales}\right)$$

donde p_{eff} es el número efectivo de procesadores que el algoritmo puede usar. Puede ser menor que el número físico de núcleos disponibles debido al número de buckets, el ancho de banda de memoria, el ancho de banda del sistema de archivos, el sobrecosto del planificador y las dependencias entre etapas del pipeline.

Para el diseño actual de entrenamiento basado en buckets:

$$p_{eff} \leq B$$

Esto se debe a que la etapa principal de entrenamiento crea una gran tarea paralela por bucket. Si $B = 1024$, la etapa de entrenamiento puede exponer naturalmente hasta unas 1024 tareas de bucket paralelas. Si el HPC proporciona menos de 1024 ranuras de trabajador, el clúster puede usarse de forma eficiente. Si el HPC proporciona muchas más de 1024 ranuras de trabajador, los procesadores adicionales no reducirán mucho el tiempo de entrenamiento a menos que aumentemos el número de buckets o añadamos otro nivel de paralelismo dentro de cada bucket, por ejemplo dividiendo el trabajo por ID o por grupos de `doy`.

Una regla práctica para un despliegue HPC es:

$$B \geq 4p$$

donde p es el número de ranuras de trabajador planificadas para la ejecución. Esto le da al planificador suficientes tareas de bucket para balancear trabajadores lentos y rápidos. Sin embargo, B no debe hacerse arbitrariamente grande porque demasiados buckets pequeños crean muchos archivos pequeños y aumentan el sobrecosto de planificación.

Puntos de partida de ejemplo:

Ranuras de trabajador planificadas	Número de buckets sugerido
64	512 o 1024
128	1024 o 2048
256	2048 o 4096
512	4096 o 8192

Por tanto, para un HPC universitario, el proyecto debe evaluarse usando ambos:

- el trabajo total, que describe el tamaño del cómputo científico; y
- el tiempo de ejecución paralelo, que describe cuánto tiempo de reloj puede reducirse usando el clúster.

La aceleración esperada es fuerte mientras p_{eff} aumenta, pero acaba saturándose debido al ancho de banda de E/S, el sobrecosto del planificador, el número de buckets y la dependencia recursiva del pronóstico sobre el horizonte H .

19 Notas Prácticas para Esta Máquina

Con unos 300 mil IDs, 40 años y 32 hilos lógicos de CPU, la mejor estrategia es mantener el número de buckets mucho mayor que el número de trabajadores. Por ejemplo, $B = 1024$ da aproximadamente $N/B \approx 293$ IDs por bucket, lo que mantiene cada tarea de bucket lo bastante grande como para amortizar el sobre costo pero lo bastante pequeña como para caber en memoria.

19.1 El hardware es simplemente más procesadores

La complejidad anterior es agnóstica al hardware: el trabajo total $W = O(NDYh)$ es el mismo tanto si el pipeline corre en núcleos de CPU como en una GPU. El hardware cambia solo dos cosas: el número de procesadores paralelos p que expone, y el factor constante c de un solo ajuste local. El tiempo de ejecución paralelo es siempre

$$T_p = c \cdot \frac{W}{p_{eff}} + \text{sobre costo}.$$

Los mínimos cuadrados ponderados por (ID, doy) son vergonzosamente paralelos: hay unos $N * D$ (aproximadamente 1.1×10^8) ajustes independientes, cada uno una pequeña regresión ponderada de $F = 5$ características sobre $W = O(Yh)$ filas de vecindad. Esto se mapea igual de bien a:

- **CPU:** muchos procesos trabajadores de bucket de un solo hilo (uno por núcleo), donde la estructura de buckets acota $p_{eff} \leq B$; o
- **GPU:** un dispositivo que evalúa miles de estos ajustes diminutos concurrentemente una vez que se agrupan en lotes, ensamblando las ecuaciones normales ponderadas $F \times F (X^{\text{top } W} X)^{\beta} = X^{\text{top } W} y$ para muchos pares (ID, doy) y resolviéndolos en una sola llamada batched.

Dos niveles de paralelismo — y cuál llena una GPU. Vale la pena ser preciso sobre qué significa aquí “más procesadores”, porque los dos hardwares añaden paralelismo en niveles diferentes:

- *Nivel externo (bucket)* — cuántos buckets corren a la vez. Esto es lo que $\min(p, B)$ acota. En CPU, p es el número de procesos trabajadores ($\approx$ núcleos); en GPU, p es el número de **GPUs** (un trabajador *dask-cuda* por dispositivo), a menudo solo 1–8.
- *Nivel interno (ajuste)* — cuántos de los $\approx (N/B) \cdot D \approx 10^5$ ajustes independientes de un bucket se evalúan juntos. En CPU esto es 1 (cada trabajador ajusta uno a la vez, BLAS fijado). En GPU esto es el **ancho de lote**: el dispositivo factoriza 10^5 sistemas 5×5 apilados a la vez. $\min(p, B)$ **no** acota este nivel.

Concretamente, una sola A100 (108 SMs, hasta ~ 2048 hilos residentes cada uno, así que $\approx 2.2 \times 10^5$ hilos en vuelo) recibe los $\approx 10^5$ ajustes de un bucket entero por oleada y los despeja en milisegundos. Así que una A100 da $p = 1$ a nivel de bucket pero $\sim 10^5$ carriles de ajuste por oleada — la velocidad reside en el nivel interno. Esto también invierte la guía sobre el número de buckets: la CPU quiere **muchos** buckets ($B \geq 4p$) para mantener los núcleos ocupados, mientras que la GPU quiere buckets **lo bastante grandes** como para que cada lote llene el dispositivo, así que B no se empuja alto en ejecuciones con GPU.

Así que una GPU es, en términos PRAM, simplemente una fuente de un p efectivo mayor (con una constante c diferente): un p externo modesto multiplicado por un lote interno muy grande. Cuál de CPU o GPU produce el menor $c \cdot T_p$ para esta carga de trabajo es una pregunta empírica, respondida por los experimentos de escalamiento de más abajo, no por el modelo de complejidad.

Las aceleraciones restantes, independientes del hardware, siguen viniendo de:

- evitar escaneos repetidos de parquet (el rediseño bucket-año),
- usar archivos bucket-año,
- mantener acotada la memoria del trabajador,
- paralelizar por bucket,
- y reparar solo los DOYs fallidos en lugar de repetir todo el entrenamiento.

20 Metodología Experimental

Esta sección define cómo se evalúa la implementación en el HPC universitario. El objetivo es medir, como función del número de procesadores p , tres cantidades estándar de rendimiento paralelo, y hacerlo tanto en hardware de CPU como de GPU y en ambas versiones del dataset.

20.1 Cantidades medidas

Para un tamaño de problema fijo, sea $T(p)$ el tiempo de reloj de una fase (o del pipeline de extremo a extremo) usando p procesadores. Reportamos:

$$\text{Tiempo de ejecución: } T(p) \quad \text{Aceleración: } S(p) = \frac{T(1)}{T(p)} \quad \text{Eficiencia: } E(p) = \frac{S(p)}{p}$$

En términos sencillos: la **aceleración** responde “¿cuántas veces más rápido lo hicieron p trabajadores?” y la **eficiencia** responde “¿qué fracción del tiempo de esos trabajadores fue realmente útil?”. La aceleración ideal es $S(p) = p$ (8 trabajadores $\rightarrow$ 8× más rápido) y la eficiencia ideal es $E(p) = 1$ (100% de cada trabajador ocupado). La eficiencia real cae por debajo de 1 porque parte del trabajo es secuencial o los trabajadores se esperan entre sí o al disco. $S(p)$ es idealmente lineal y $E(p)$ es idealmente 1. El análisis PRAM predice la desviación de lo ideal: como las etapas de entrenamiento y pronóstico son bucket-paralelas con $p_{\text{eff}} \leq B$, la eficiencia se mantiene alta mientras $p \leq B$ y luego se degrada; el ancho de banda de E/S, el sobre costo del planificador y las dependencias secuenciales del pipeline fijan el techo práctico.

20.2 Definición de un “procesador” p

Para comparar CPU y GPU en un mismo eje, p cuenta ranuras de trabajador independientes:

Hardware	Un procesador (unidad p)	Cómo se varía p
CPU (SLURM)	un proceso trabajador de Dask de un solo hilo (un núcleo, BLAS fijado a 1 hilo)	<code>dask_jobqueue.SLURMCluster</code> escalado a p trabajadores
GPU (A100)	una GPU A100 (un trabajador <code>dask-cuda</code>)	p = número de A100s en la asignación

Dentro de una sola GPU, el solucionador batched de mínimos cuadrados ponderados usa todos los núcleos CUDA; ese paralelismo intra-dispositivo se mantiene fijo y se absorbe en la constante c , así que el eje p reportado es el número de GPUs. Esto mantiene $S(p)$ y $E(p)$ significativos a través del hardware: ambos responden “¿cuánto más rápido con p trabajadores independientes?”.

20.3 Controles

Para hacer las mediciones reproducibles y justas:

- **La línea base $T(1)$** es un solo trabajador con `threads_per_worker = 1` y los pools de hilos de BLAS fijados a 1 (`OMP_NUM_THREADS=1`, etc.), como en `run_pipeline.sh`.
- **El número de buckets** se mantiene en $B \leq 4, p_{\text{max}}$ durante todo el barrido para que la granularidad de buckets nunca sea el factor limitante antes de que p alcance su máximo.
- **Ensayos repetidos**: cada punto (p, n) se ejecuta k veces (p. ej., $k = 3$) y se reporta el tiempo de reloj mediano, para suprimir el jitter del sistema de archivos y del planificador.
- **Almacenamiento**: el scratch / spill del trabajador va a almacenamiento rápido local del nodo (`local_directory = $TMPDIR`); el parquet de entrada vive en el sistema de archivos compartido.
- **Caché caliente vs fría** se registra; las ejecuciones con caché fría (primer acceso) se reportan por separado porque las fases de preprocesamiento limitadas por E/S son sensibles a la caché.
- Solo se cronometra la fase bajo prueba (con `time.perf_counter` alrededor de la llamada al driver); las entradas de fases anteriores se precaculan una vez y se reutilizan.

Los dos estudios de escalamiento de abajo plantean dos preguntas diferentes. El **escalamiento fuerte** fija el problema y añade trabajadores — “¿puede el clúster hacer que *este* trabajo termine antes?”. El **escalamiento débil** hace crecer el problema al ritmo de los trabajadores — “si duplicamos tanto los datos como las máquinas, ¿se mantiene plano el tiempo?”. El escalamiento fuerte expone el techo $p_{\text{eff}} \leq B$; el escalamiento débil aísla el sobrecosto que crece con el número de trabajadores.

20.4 Escalamiento fuerte (problema fijo, más procesadores)

Mantener el tamaño del problema fijo en la grilla completa (o en un subconjunto representativo fijo de N IDs) y variar p :

$$p \in \{1, 2, 4, 8, 16, 32, \dots\} \quad (\text{CPU}) \quad p \in \{1, 2, 4, 8\} \quad (\text{GPUs A100})$$

Para cada p registramos $T(p)$ para la fase dominante de **entrenar-coeficientes** (y, por separado, las fases de construir-bucket y climatología limitadas por E/S, y opcionalmente el pronóstico). A partir de estas calculamos $S(p)$ y $E(p)$ y graficamos las tres contra p . La curva esperada sube casi linealmente mientras $p \leq B$ y la carga de trabajo es limitada por cómputo, luego se aplana — probando directamente la predicción $p_{\text{eff}} \leq B$.

20.5 Escalamiento débil (más procesadores, problema proporcionalmente mayor)

Mantener constante el trabajo por procesador escalando el número de IDs entrenados con p :

$$n(p) = n_0 \cdot p$$

donde n es el número de IDs entrenados. El subconjunto de IDs se selecciona de forma determinista mediante el parámetro `bucket_ids` existente (cada bucket contiene $\approx N/B$ IDs, así que elegir $\lceil 4p \rceil$ buckets da un $n(p)$ controlado). La eficiencia de escalamiento débil es

$$E_{\text{weak}}(p) = \frac{T(1, n_0)}{T(p, n_0 p)}$$

que es idealmente 1. Como los ajustes por (`ID`, `doy`) son independientes, se espera que la parte de cómputo del entrenamiento escale débilmente bien; las desviaciones aíslan el sobrecosto de E/S y de planificación que crece con el número de trabajadores.

20.6 Datasets: PISCOT v1.0 vs v2.0

El protocolo completo de escalamiento fuerte y débil se ejecuta en ambas versiones del dataset. Las dos versiones se suministran al mismo pipeline mediante los parámetros de ruta de entrada / carpeta de variables (`--base`, `--td-var`, `--tmin-var`) escribiendo a raíces de resultados separadas, así que no difiere ningún código entre ejecuciones. Comparamos:

- **Rendimiento:** curvas $T(p)$, $S(p)$, $E(p)$ para v1.0 vs v2.0 (el trabajo W difiere solo si N , Y o la densidad de datos difieren entre versiones).
- **Ciencia:** distribuciones de los coeficientes ajustados y de la habilidad predictiva, para evaluar cómo la versión del dataset afecta el modelo estimado, independientemente del tiempo de ejecución.

20.7 Configuraciones de hardware

Configuración	Clúster	p barrido
CPU multinodo	<code>dask_jobqueue.SLURMCluster</code> (un proceso por núcleo, $B \geq 4p_{\text{max}}$)	procesos trabajadores
GPU una/varias A100	<code>dask-cuda</code> (<code>LocalCUDACluster</code> o <code>SLURM --gres=gpu:a100:p</code>)	número de A100s

20.8 Reportes

Para cada combinación (dataset $\times$ hardware) producimos: (1) una tabla de $T(p)$, $S(p)$, $E(p)$ para el barrido de escalamiento fuerte; (2) una tabla de $E_{\text{weak}}(p)$; y (3) tres gráficas — tiempo de ejecución vs p , aceleración vs p (con la referencia ideal $S(p)=p$) y eficiencia vs p . Estas cuantifican tanto cómo escala la implementación como qué hardware minimiza el tiempo de reloj para esta carga de trabajo.